

SYSTEM FLOWCHARTS

Ten diagrams of how the game works, from the moment-to-moment turn to the long-term collection. Each one mirrors the shipped code; the source file is named on its page.

01 · CORE AND META LOOPS

How a single level feeds the long-term collection, and what brings the player back.

02 · THE FOOL'S JOURNEY

How the 85 levels are structured into chapters, gates and Pip's scenes.

03 · SCREEN FLOW

Every screen and panel, and how the player moves between them.

04 · TURN RESOLUTION

What happens between the player's swipe and the board settling.

05 · SPECIAL TILE CREATION

Which special a matched shape makes, and where it appears.

06 · LEVEL END: WIN, LOSE AND RESCUE

How a level ends, how the reward is decided, and the extra-moves offer.

07 · BOOSTERS AND REFILLS

Using a booster, arming Tower Strike, and the refill hook when one runs out.

08 · CARD UNLOCK RULES

Which card a win grants. Majors in journey order, suits random, never a duplicate.

09 · DAILY DRAW

The once-a-day ritual that draws from the player's own collection.

10 · CODE ARCHITECTURE

Rules in pure C#, MonoBehaviours only for display, input and animation.

COLOUR KEY

■ Entry / exit point

■ Failure state

■ Special tile / effect

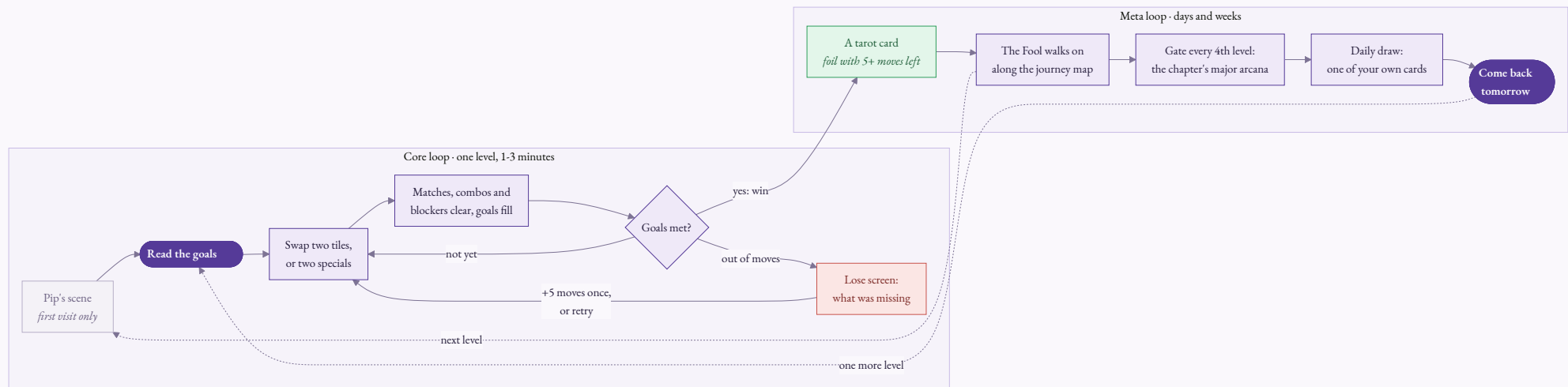
■ Reward or success

■ Monetisation hook

■ Neutral / no-cost path

CORE AND META LOOPS

How a single level feeds the long-term collection, and what brings the player back.



1 The core loop is one level of 18 to 24 moves.

2 Losing never ends the loop: rescue or retry.

3 Every win moves the Fool on the map; gates award the majors, and the daily draw has no streak to lose.

Entry / exit point

Reward or success

Failure state

Monetisation hook

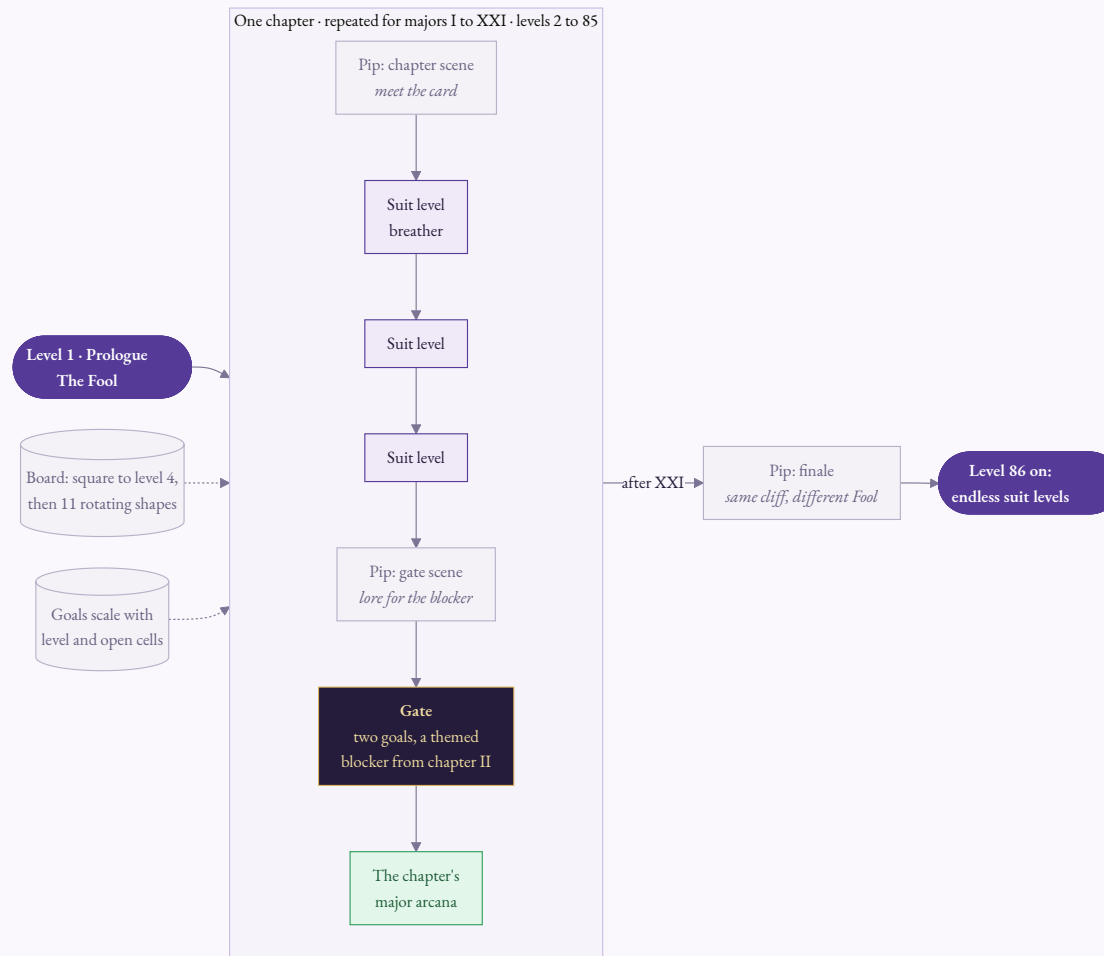
Special tile / effect

Neutral / no-cost path

Source: [GameApp.cs](#), [LevelSession.cs](#)

THE FOOL'S JOURNEY

How the 85 levels are structured into chapters, gates and Pip's scenes.



1 21 chapters of 3 suit levels and a gate; each gate awards its major, in order.

2 Pip's scenes play once, before the level: chapter scenes, gate lore, and four mechanic tips.

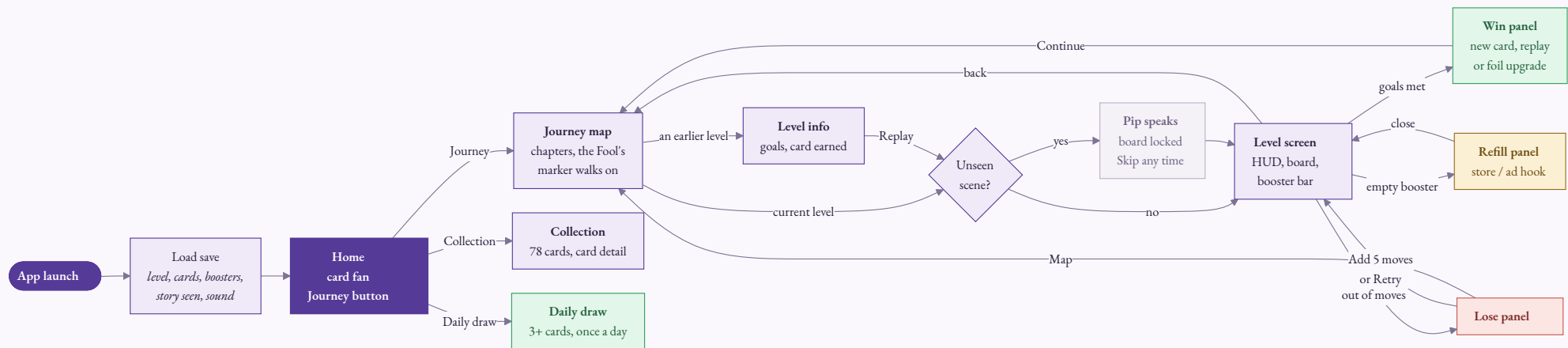
3 Every level is generated from rules and checked by the balance simulator.

■ Entry / exit point
 ■ Failure state
 ■ Special tile / effect
 ■ Reward or success
 ■ Monetisation hook
 ■ Neutral / no-cost path

Source: [Journey.cs](#), [BoardShapes.cs](#), [StoryTriggers](#)

SCREEN FLOW

Every screen and panel, and how the player moves between them.



1 Home, the journey map and the level; everything else is a panel.

2 Levels are entered and left through the map, so progress is always visible; Home is one tap back from the map.

3 Gold nodes are monetisation hooks; green are rewards.

Entry / exit point

Failure state

Special tile / effect

Reward or success

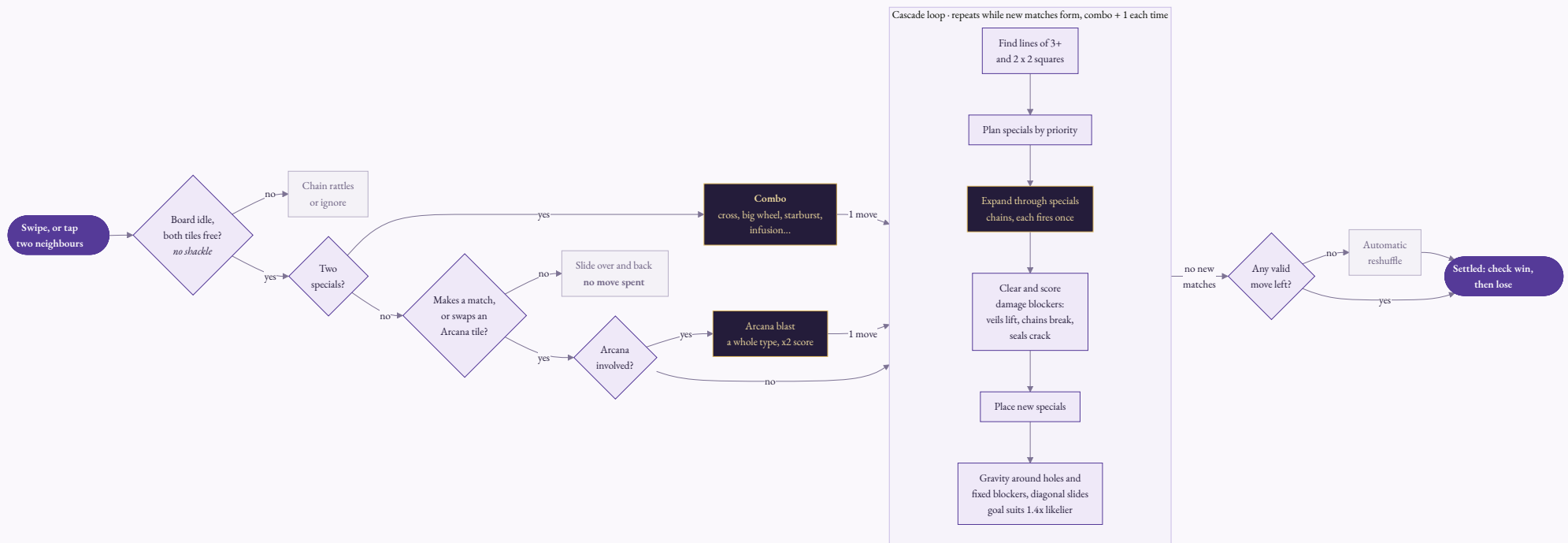
Monetisation hook

Neutral / no-cost path

Source: [GameApp.cs](#),
[JourneyMapScreen.cs](#),
[GuideOverlay.cs](#)

TURN RESOLUTION

What happens between the player's swipe and the board settling.



1 Resolved instantly in pure C#; the view replays the steps.

2 Invalid swaps are free; two specials always combine.

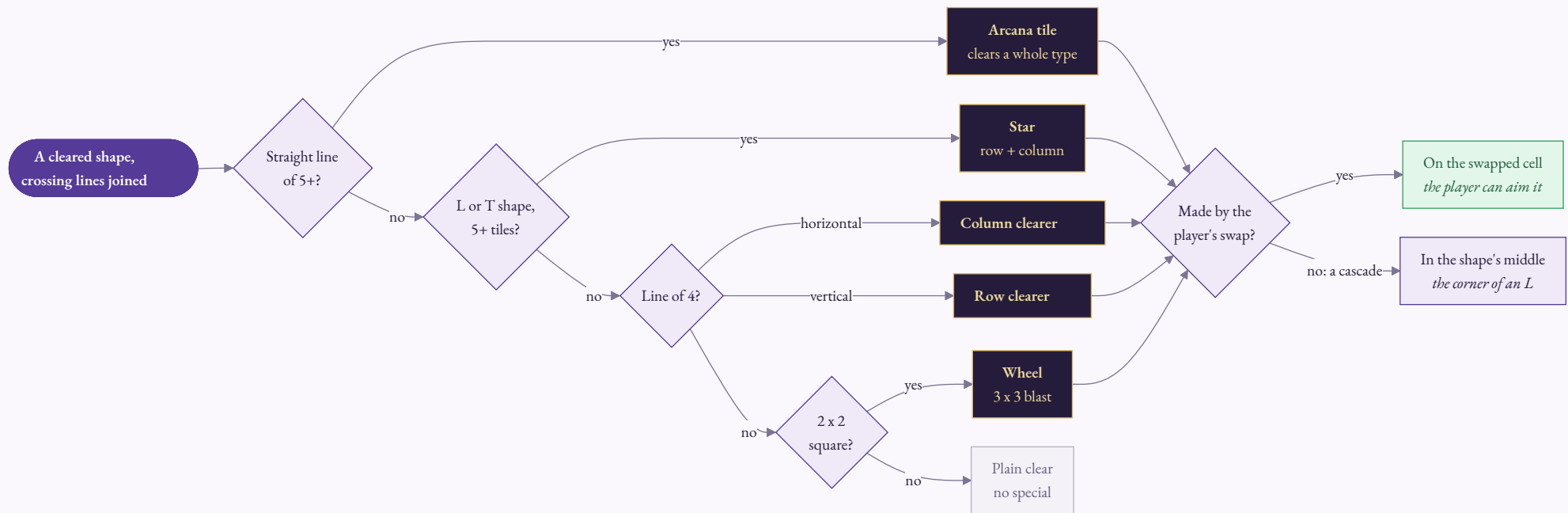
3 Clears damage blockers; gravity routes tiles around holes and seals so every cell refills.

- Entry / exit point
- Failure state
- Special tile / effect
- Reward or success
- Monetisation hook
- Neutral / no-cost path

Source: [BoardResolver.cs](#),
[StepPlayer.cs](#)

SPECIAL TILE CREATION

Which special a matched shape makes, and where it appears.



1 Checked from rarest to most common, so a harder shape is never downgraded.

2 Player-made specials can be aimed.

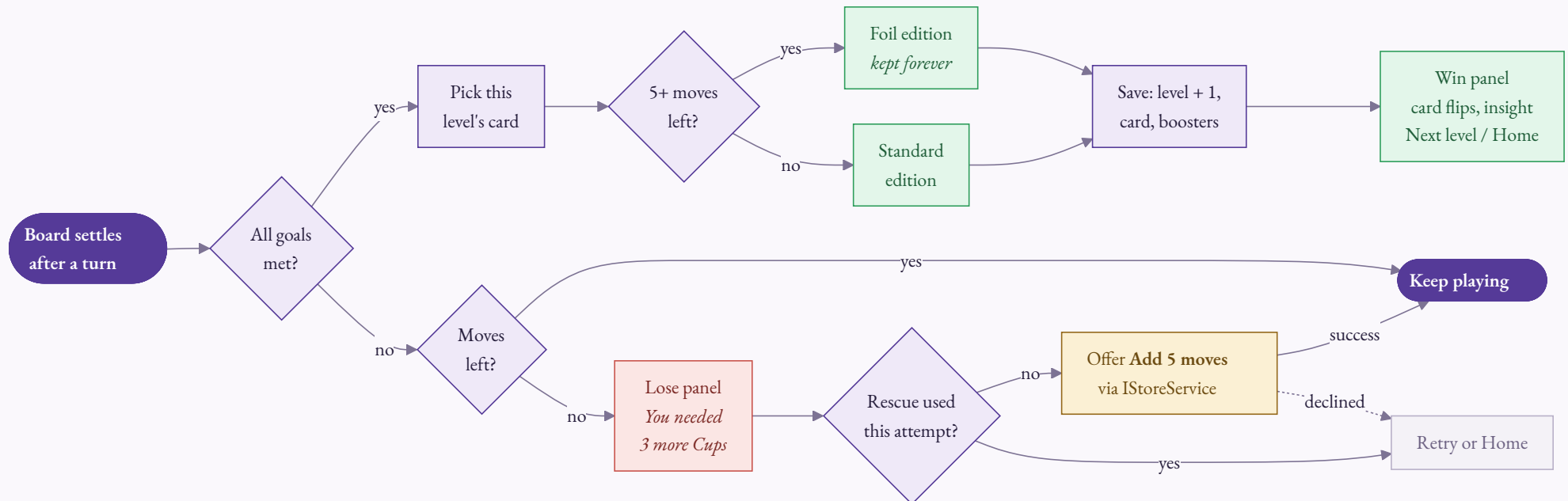
3 Specials chain; each fires once per chain.

- Entry / exit point
- Failure state
- Special tile / effect
- Reward or success
- Monetisation hook
- Neutral / no-cost path

Source: [BoardResolver.PlanSpecials](#)

LEVEL END: WIN, LOSE AND RESCUE

How a level ends, how the reward is decided, and the extra-moves offer.



1 Win is checked before lose.

2 Foil for 5+ moves left; it is never lost.

3 The rescue is offered once per attempt, through the store interface.

Entry / exit point

Reward or success

Failure state

Monetisation hook

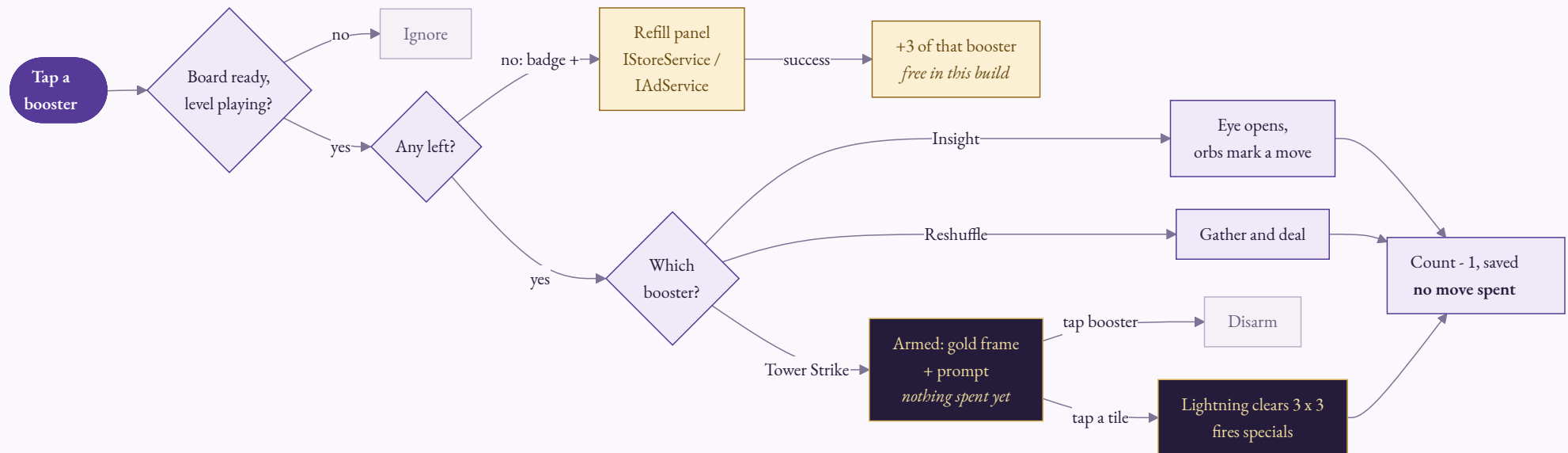
Special tile / effect

Neutral / no-cost path

Source: [LevelSession.cs](#),
[GameApp.OnWon / OnLost](#)

BOOSTERS AND REFILLS

Using a booster, arming Tower Strike, and the refill hook when one runs out.



1 No booster costs a move.

2 Tower Strike must be armed first, so a stray tap never wastes it.

3 Refills go through IStoreService / IAdService; free in this build.

Entry / exit point

Reward or success

Failure state

Monetisation hook

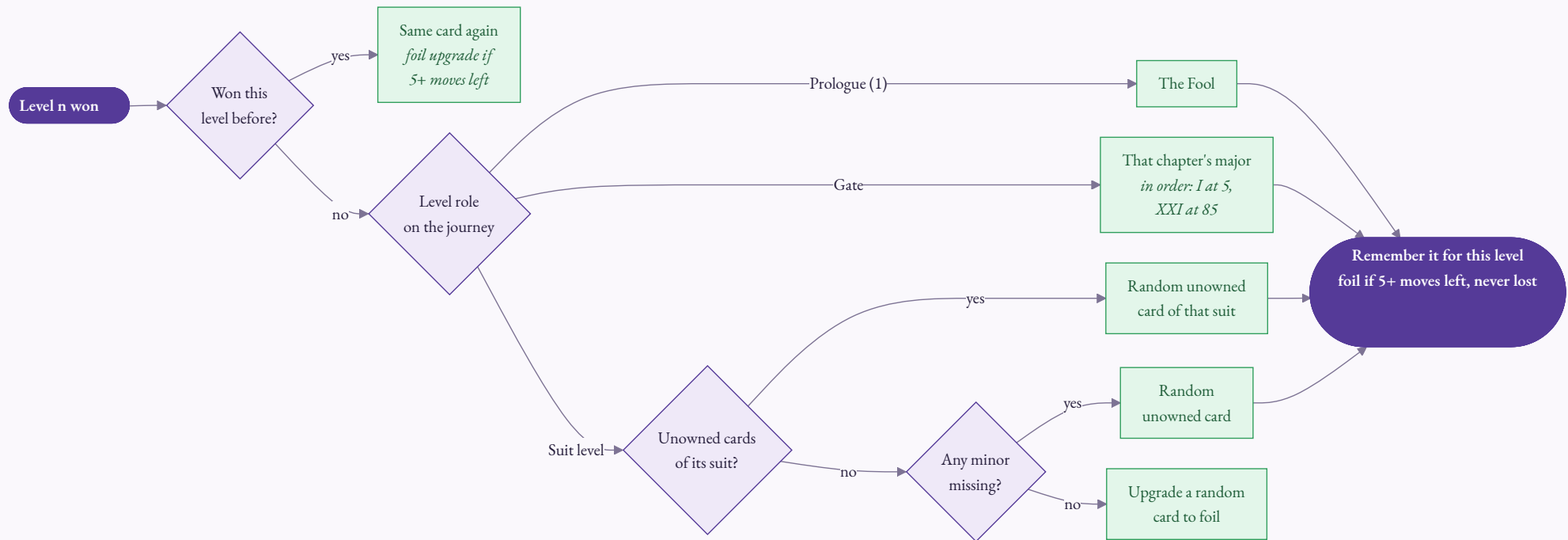
Special tile / effect

Neutral / no-cost path

Source: `BoosterController.cs`,
`BoosterInventory.cs`

CARD UNLOCK RULES

Which card a win grants. Majors in journey order, suits random, never a duplicate.



1 The Fool starts the journey; each gate gives its chapter's major.

2 A level always keeps its card: replays upgrade it to foil.

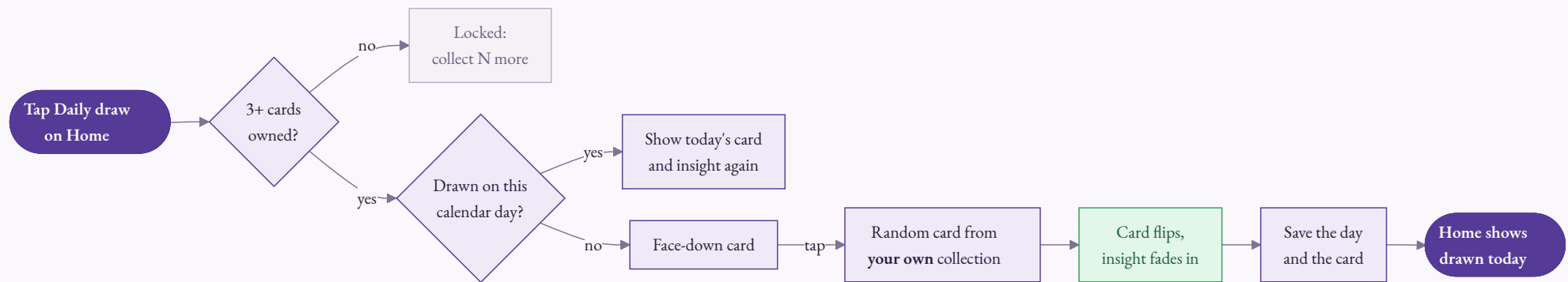
3 The journey completes the deck with no duplicates (an automated test plays it).

Entry / exit point
 Failure state
 Special tile / effect
 Reward or success
 Monetisation hook
 Neutral / no-cost path

Source: [LevelRewards.cs](#),
[CardUnlockRules.cs](#)

DAILY DRAW

The once-a-day ritual that draws from the player's own collection.



1 Unlocks at 3 cards.

2 One draw per calendar day; nothing is lost by skipping a day.

3 The bigger the deck, the more varied the draw.

■ Entry / exit point

■ Reward or success

■ Failure state

■ Monetisation hook

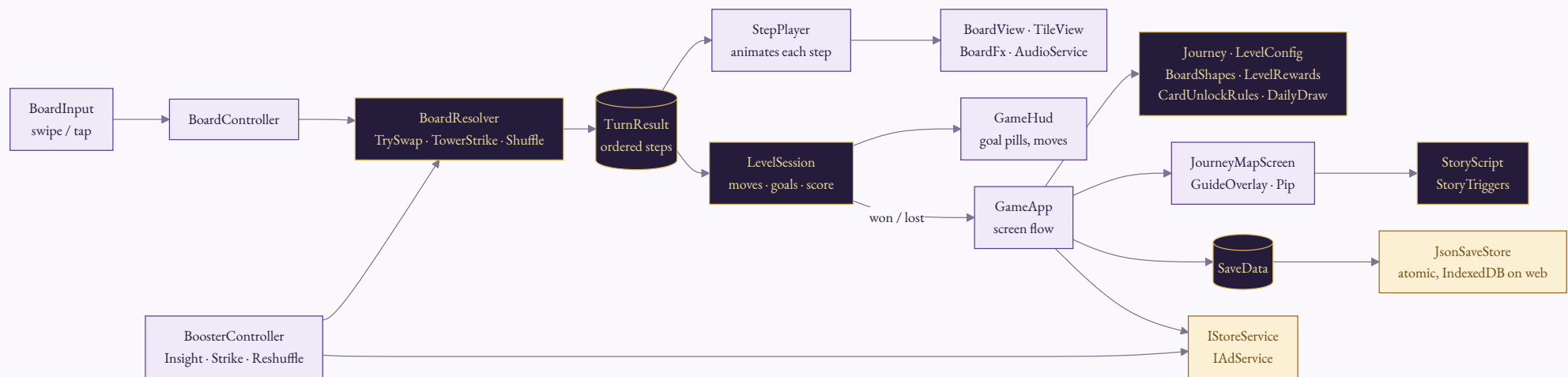
■ Special tile / effect

■ Neutral / no-cost path

Source: [DailyDraw.cs](#),
[DailyDrawPanel.cs](#)

CODE ARCHITECTURE

Rules in pure C#, MonoBehaviours only for display, input and animation.



1 The model never waits for animation: a move returns a list of steps.

2 Randomness is injected, so tests and the balance bot are repeatable.

3 Colour key: pale = Game (MonoBehaviours), dark = Core (pure C#, tested), gold = services behind interfaces.

Entry / exit point
 Failure state
 Special tile / effect
 Reward or success
 Monetisation hook
 Neutral / no-cost path

Source: [Docs/ARCHITECTURE.md](#)